Project Technical Report

Defending Against Data Reconstruction Attacks in Federated Learning

Evaluating Activation Clustering Under Gradient-Reconstructed Data

Submitted by

Student ID	Name
230170409	Nazanin Parvizi
230155722	Sukirat Singh Dhillon
230171256	Simon Kraft

1. Dataset Description

1. Overview

The project uses the MNIST handwritten digit dataset as its primary data source, with the Fashion-MNIST dataset used for the federated learning component. MNIST consists of 70,000 grayscale 28×28 images of the digits 0–9, split into 60,000 training and 10,000 test samples, with 6,000 examples per class. Fashion-MNIST has an identical structure but depicts 10 categories of clothing items. Both datasets are balanced across classes, requiring no re-sampling.

- MNIST is used for both the backdoor attack experiments and the DLG gradient-inversion experiments. It is the dataset being reconstructed by the DLG attack and subsequently analyzed by the Activation Clustering defense.
- Fashion-MNIST is used to train a federated learning model using the Flower framework, providing the realistic federated context in which gradient leakage attacks like DLG are a practical threat.

2. Data Preprocessing & Feature Engineering

2.1 MNIST Normalization

For all MNIST experiments, images were normalized using the standard MNIST channel mean (0.1307) and standard deviation (0.3081) via PyTorch's transforms.Normalize. This maps the raw [0, 1] pixel range to a standardized distribution centered near zero. In Naz's pipeline, the cleaned MNIST training set was loaded from a saved .pt file and additionally handled potential format variations (dictionary, tuple, or TensorDataset) and ensured images were in float32 [0, 1] range with an explicit channel dimension.

2.2 Federated Partitioning

For the Flower-based federated simulation, the Fashion-MNIST training set was partitioned across clients using an IID (Independent and Identically Distributed) strategy provided by the flwr-datasets library's IidPartitioner. Each client's local partition was then split 80/20 into local training and validation sets using a fixed random seed of 42, ensuring reproducibility across runs.

2.3 Backdoor Trigger Injection

We implemented a custom Poisoned Dataset class to simulate a trigger-based backdoor attack. The trigger configuration was consistent across experiments:

- Trigger pattern: A 3×3 pixel square patch injected at position (24, 24) — the bottom-right corner of the 28×28 image.
- Trigger value: 2.8, deliberately chosen to be out-of-distribution for MNIST (which occupies [0, 1]), making the pattern strongly distinctive for the model to learn.
- Target class: Class 1 (the digit '1'). Any non-target-class sample carrying the trigger is relabeled as class 1.
- Poison rate: 20% of all non-target-class training samples were poisoned in one of the pipelines; and the other used 30%.

The slight difference in poison rates reflects the two different experimental orderings: first pipeline poisoned original MNIST images before training, while the other applied the backdoor to

a smaller, already-reconstructed dataset, requiring a higher rate to produce sufficient poisoned samples.

2.4 DLG-Reconstructed Data

A key preprocessing step unique to this project is the generation of gradient-reconstructed datasets. Both pipelines used the DLG attack (described in Section 3) to produce a reconstructed version of training images from their corresponding gradient signals. These reconstructed images are imperfect approximations of the originals, exhibiting varying reconstruction quality depending on convergence. Second pipeline explicitly filtered out samples with NaN final gradient distances before saving the reconstructed dataset.

3. Methodology / Modeling Approach

3.1 Background: The Activation Clustering Defense

The central goal of this project is to evaluate the robustness of the Activation Clustering (AC) methodology introduced by Chen et al. in *Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering*. The AC defense works on a core observation; because backdoored samples are trained to produce a specific model output regardless of their true content, they activate the model's internal layers in a systematically different way than clean samples. When the neural activations of all target-class samples are clustered (using K-Means with k=2), backdoored and clean samples are expected to form two separable groups.

This project examines if the AC defense remains effective when the dataset used for clustering has been reconstructed via a DLG gradient-inversion attack rather than being the original data? This is a realistic and important scenario in federated learning, where a server that intercepts client gradients could attempt to reconstruct training samples and then check them for backdoor signatures, without ever having direct access to the client's private data.

3.2 First Approach — Backdoor First, Then Reconstruct

This approach follows the order: backdoor attack → model training → DLG reconstruction → AC evaluation.

Step 1 Backdoor Attack: A PoisonedDataset is created by injecting the trigger into 20% of non-target-class MNIST training samples. A MidCNN model is then trained on a poisoned subset of 500 samples for 10 epochs using Adam with cosine annealing. This produces a backdoored model that has learned to associate the trigger pattern with class 1.

Step 2 DLG Reconstruction: It collects 100 target-class (class 1) samples from the poisoned training set, comprising a mix of genuinely clean class-1 images and backdoored non-class-1 images that were relabeled. A ReconstructedDataset class is used to reconstruct every one of these 100 samples from their gradient signals using 150 L-BFGS iterations of the DLG attack. Crucially, the gradients are computed using the trained backdoor model, so the reconstructed images carry the model's learned representation of those samples, including, to some degree, the trigger pattern.

Step 3 AC Evaluation: Activations are extracted from every hooked layer of the backdoor model for both the original 100 samples and their 100 reconstructed counterparts. Four metrics are computed per layer to score how well clean and poisoned samples separate in activation space: Silhouette score, Fisher ratio, K-Means purity, and LDA accuracy. A composite separability score is used to identify the best and worst separating layers. 2D projections (PCA, t-SNE, LDA) are visualized for the best, penultimate, and worst layers. Additionally, it compares the clean model's activations to the backdoor model's activations at the best layer to confirm that the observed cluster separation is caused by the backdoor rather than by natural digit structure.

3.3 Second Approach — Reconstruct First, Then Backdoor

This pipeline reverses the ordering: DLG reconstruction → backdoor attack → model training → AC evaluation.

Step 1 DLG Reconstruction: Starting from a cleaned MNIST training set, it trains a SimpleInstrumentedCNN on clean data. The DLG attack is then applied to reconstruct 80 samples from their gradients using 300 L-BFGS iterations. The result is saved as a reconstructed_mnist.pt file. Samples with NaN final gradient distances (failed convergence) are filtered out before saving.

Step 2 Backdoor Attack on Reconstructed Data: The poisoning is applied directly to the reconstructed images, not to original MNIST samples. A 30% poison rate is used, with the same trigger configuration as First Approach. This simulates a more adversarial scenario, where an attacker who only has access to gradient-reconstructed data (not the original training set) injects a backdoor and retrains the model.

Step 3 Model Training: Two SimpleInstrumentedCNN models are trained, a clean model on standard MNIST and a backdoor model on the poisoned reconstructed dataset. The backdoor model is trained for 20 epochs due to the small size of the reconstructed training set (~60 samples after filtering).

Step 4 AC Evaluation: Here used ICA (Independent Component Analysis with 10 components) rather than PCA as the dimensionality reduction step before K-Means clustering. The same four separability metrics are computed per layer. Confusion matrices, 2D projections (ICA, t-SNE, LDA), and per-layer detection metrics (precision, recall, F1, silhouette) are reported. A clean model vs. backdoor model comparison is also included to validate the AC finding.

3.4 Federated Learning Environment

We Implemented the federated learning simulation using the Flower (flwr) framework, establishing the realistic threat context that motivates the entire project. In federated learning, clients train models locally and share only gradient updates with a central server, they never share raw data. This creates a direct opportunity for a gradient inversion attack, where the server (or a man-in-the-middle adversary) can apply DLG to intercepted gradient updates to attempt reconstruction of clients' private training images. This implementation covers:

- A ClientApp (client_app.py) that handles local training and evaluation, receiving model weights from the server and returning updated gradients along with training metrics.
- A ServerApp (server_app.py) that orchestrates FedAvg aggregation across clients, runs a global evaluation function after each round, and saves the final aggregated model.
- A task module (task.py) defining the model architecture (Net), the IID data partitioning logic, and the training and evaluation functions shared across client and server.
- A pyproject.toml configuration specifying 10 federated rounds, 3 local epochs per client, a learning rate of 0.01, and a batch size of 64.

4. Baseline Model(s)

4.1 Net — Federated Learning Baseline

The baseline model for the federated learning component is a simple CNN (Net) adapted from the PyTorch 60-Minute Blitz tutorial, trained on Fashion-MNIST:

- Conv1: 1 → 6 channels, 5×5 kernel, followed by MaxPool 2×2
- Conv2: 6 → 16 channels, 5×5 kernel, followed by MaxPool 2×2
- FC1: 16×4×4 → 120; FC2: 120 → 84; FC3: 84 → 10

Trained with SGD (lr=0.01, momentum=0.9) for 3 local epochs per round across 10 server rounds with a batch size of 64. This architecture is intentionally simple: lightweight models produce more informative per-sample gradients, which makes them more susceptible to the DLG attack and thus a meaningful baseline for gradient leakage analysis.

4.2 SimpleInstrumentedCNN — Baseline for DLG and Backdoor attack testing

In the second pipeline we used a 4-layer CNN with forward hooks on every layer as the starting model for DLG reconstruction experiments. The architecture is: Conv1 (1→16, 3×3, padding 1) → MaxPool → Conv2 (16→32, 3×3, padding 1) → MaxPool → FC1 (32×7×7→64) → FC2 (64→10). This model is used for both gradient computation (to produce target gradients for DLG) and, after retraining on poisoned data, for AC evaluation. Its moderate size provides a balance between reconstruction feasibility and classification performance.

4.3 Justification

Simple CNN architectures serve as the appropriate baseline for this study for two reasons. First, the DLG attack, which relies on inverting the map from gradients to inputs, is theoretically strongest against architectures with fewer parameters, where the gradient signal carries proportionally more information per training sample. Second, using architectures consistent with the federated learning literature allows the results to be interpreted in the context of real federated systems, where lightweight models are common due to communication constraints.

5. Proposed Model(s)

5.1 Instrumented CNN Architecture Suite (model.py)

A shared library of instrumented CNN models was developed with a BaseACModel parent class that provides a unified hook registration system. All models store activations by layer name in a

dictionary accessible via get_activations(), and forward hooks are registered automatically at instantiation. This design allows the AC defense to be applied identically across all architectures. Three variants were implemented:

Model	Layers	Approx. Params	Activation
SmallCNN	2 Conv (1→16→32, stride 2) + FC	~37K	ReLU
MidCNN	3 Conv (1→12→12→12, stride 2/2/1) + FC	~17K	Sigmoid / ReLU
LargeCNN	3 Conv (1→32→64→128) + Dropout(0.3) + 3 FC	~1.3M	ReLU

In the first experiments, MidCNN with ReLU activation was selected as the backdoor model for training on the poisoned dataset. SmallCNN was initialized at the start of the notebook as an exploratory model. Both share the same BaseACModel hook infrastructure, so the AC analysis pipeline works identically for either.

5.2 SimpleInstrumentedCNN (Second experiment)

It defines its own instrumented CNN independently of model.py, with equivalent hook functionality but a slightly different architecture: Conv1 (1→16) → MaxPool → Conv2 (16→32) → MaxPool → FC1 (32×7×7→64) → FC2 (64→10). This 4-layer model has ~103K parameters and uses ReLU activation throughout. The hook system stores post-activation outputs for conv1, conv2, fc1, and fc2, providing four analysis points for the AC defense.

5.3 DLG Gradient Inversion Module

A shared, reusable DLG reconstruction module was developed and used by both experiment. Key design decisions:

- Optimizer: L-BFGS with history_size=10 and max_iter=20 per closure call, which efficiently handles the highly non-convex gradient-matching objective.
- Soft label optimization: Label logits are jointly optimized with the dummy image, allowing simultaneous recovery of both the image and its class label without requiring prior knowledge of the label.
- Noise support: An optional Gaussian noise standard deviation parameter can be added to the intercepted gradients to simulate the effect of differential privacy or communication channel noise.
- Best-loss tracking: The reconstruction corresponding to the minimum gradient distance across all iterations is returned, avoiding quality degradation from late-iteration oscillation.
- Clamping: Reconstructed outputs are clamped to the valid normalized pixel range, preventing display artifacts.

6. Experimental Setup

6.1 Data Splits

MNIST: Standard 60,000/10,000 train/test split. First experiment uses a poisoned subset of 500 training samples to train the backdoor model, then selects 100 target-class samples for AC analysis. Secondexperiment reconstructs 80 samples from the full training set, filters out failed reconstructions (NaN losses), and trains on the resulting ~60 samples.

6.2 Evaluation Metrics

Four complementary metrics are computed per layer in both AC pipelines:

- Silhouette Score: Measures cluster cohesion and separation in the projected activation space. Ranges from –1 (misassigned) to +1 (perfectly separated). Higher values indicate the AC defense is more effective.
- Fisher Ratio: Ratio of between-class scatter to within-class scatter. Captures how much more variable the activations are across classes than within them — a higher ratio means more discriminative features.
- K-Means Purity: The percentage of the dominant ground-truth class (clean vs. poisoned) in each detected cluster. A purity of 1.0 means every cluster is entirely homogeneous.
- LDA Accuracy: Linear Discriminant Analysis accuracy using ground-truth labels as the supervision signal. Provides an upper-bound reference for how linearly separable clean and poisoned activations are in the reduced space.

A composite separability score (normalized mean of all four metrics) is used to rank layers and identify the best layer for AC-based detection. Detection performance is also reported as per-layer precision, recall, F1-score, and confusion matrices, comparing K-Means cluster assignments to ground-truth poison flags.

6.3 Evaluation of the Reconstruction Problem

Both experiment explicitly evaluate the quality of DLG reconstruction as a prerequisite to the AC analysis. Reconstruction quality is assessed via the final gradient distance (L2 norm between dummy and target gradients after optimization).

6.4 Clean vs. Backdoor Model Comparison

We include an experiment comparing activations from a clean model (trained on non-poisoned data) and the backdoor model (trained on poisoned data) on the same set of mixed samples. This controls for the alternative hypothesis that observed cluster separation in the AC analysis is caused by digit structure or class membership rather than by the backdoor. If separation appears only in the backdoor model and not in the clean model, it provides strong evidence that the AC defense is detecting a genuine backdoor signal.

7. Results

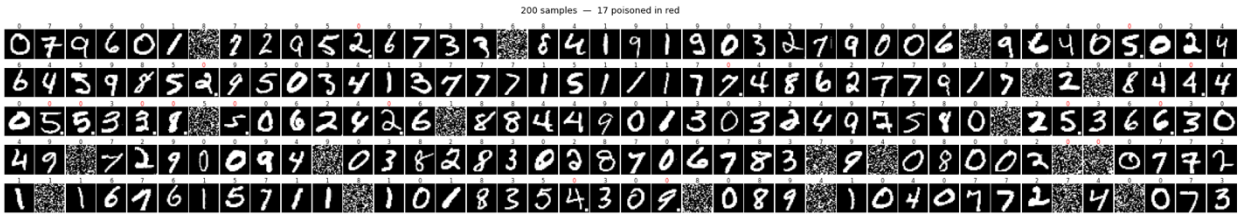
7.1 DLG Reconstruction Quality

DLG reconstruction quality is non-uniform across samples in both pipelines, which is the central complicating factor for the AC evaluation. In second pipeline, the histogram of final gradient distances over 80 reconstructed samples shows a bimodal distribution: a spike near zero (near-perfect reconstructions where gradient matching succeeded), a moderate cluster around 5–10

(partial reconstructions with visible artifacts), and a small number of outliers at very high loss values (failed reconstructions that were filtered before downstream use). This heterogeneity means the AC clustering operates on a mix of high-fidelity and low-fidelity approximations of the original images.

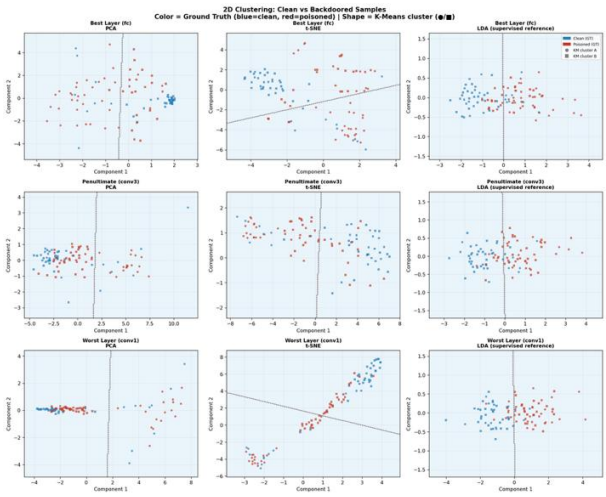
In first pipeline, reconstruction was performed using a trained backdoor model with fewer iterations (150 vs 300). Visualizations of original vs. reconstructed sample pairs, alongside pixel-wise difference maps, confirm that reconstruction quality varies by sample, some images are recognizable reconstructions of the original digit, while others retain only structural outlines. Crucially, for backdoored samples, the trigger pattern (the 3×3 white patch in the bottom-right corner) is partially visible in successful reconstructions, since the trigger was the most gradient-informative feature for those mislabeled samples.

The second pipeline turned that around and reconstructed the images first, before adding the backdoor to them, effectively simulating what would happen, if a malicious client, without its own private training data, intercepts the gradients of other benign clients, and uses the resulting reconstructed images for a backdoor attack. The results are shown here:



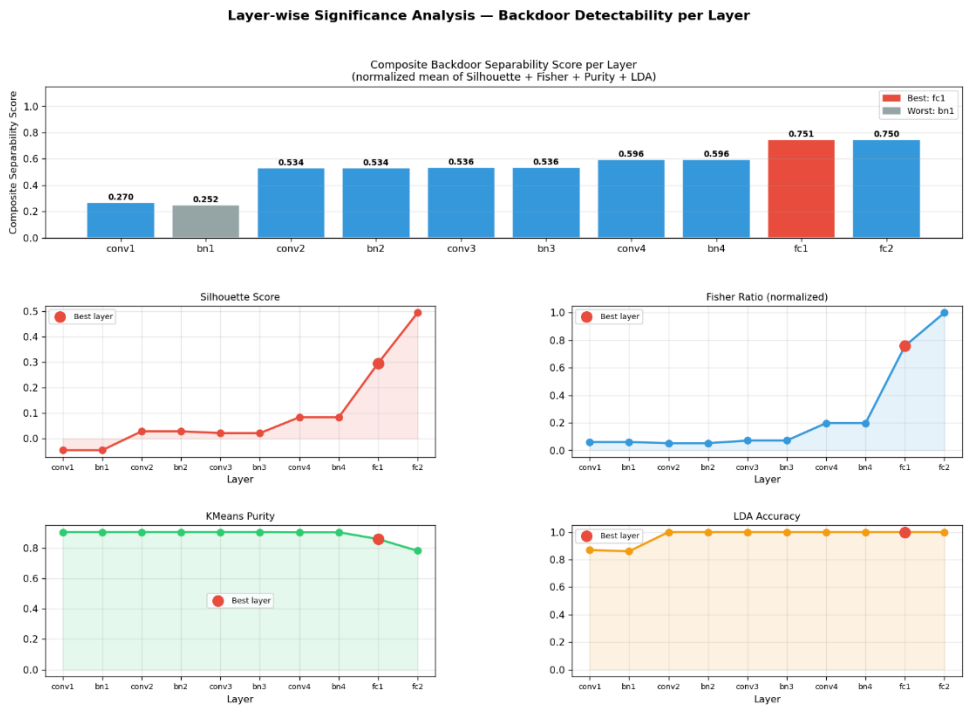
7.2 Activation Clustering on Reconstructed Data

We computed the four separability metrics across all layers of the backdoor MidCNN using activations from the 100 DLG-reconstructed target-class samples. A composite separability score was derived to identify the best and worst layers. 2D projections (PCA, t-SNE, LDA) were visualized for the best layer, the penultimate layer, and the worst layer.

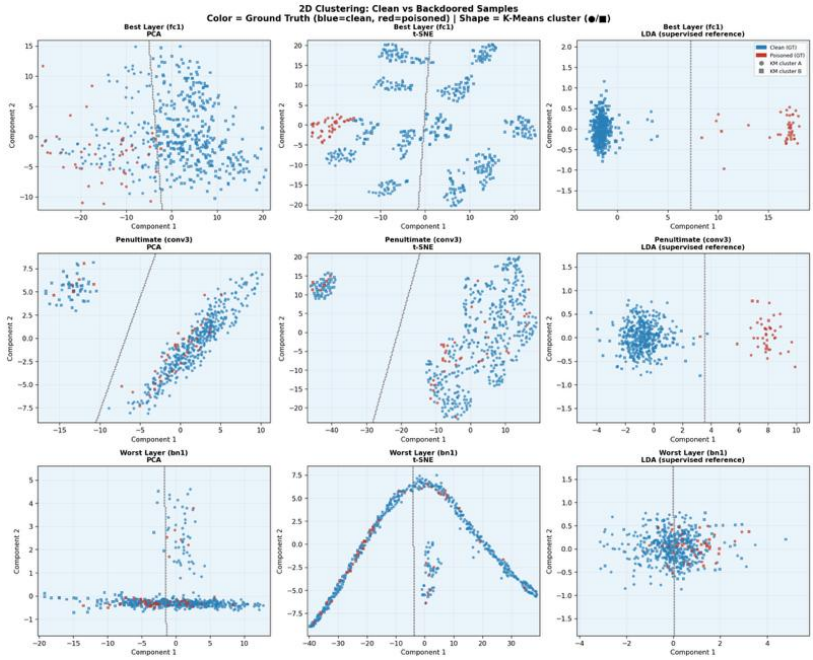


7.3 Activation Clustering on Poisoned Reconstructed Data

Second pipeline applied the AC defense to a mixed dataset consisting of poisoned reconstructed samples and clean MNIST samples, evaluated against a model trained on the poisoned reconstructed data. Per-layer, the silhouette scores, the fisher ratio, the k-means purity and the LDA-accuracy are reported and visualized.



Furthermore, again the separability on this dataset is shown.



8. Limitations & Planned Next Steps

A primary limitation of this study is the **computational intensity** of the DLG attack, which necessitated a sample size under 500 images and limited our evaluation to low-resolution datasets like MNIST. We observed that reconstruction quality significantly degrades as models approach convergence. The “vanishing gradients” in highly accurate models provide a weaker signal for the L-BFGS optimizer compared to untrained models. Furthermore, the effectiveness of the Activation Clustering (AC) defense remains highly sensitive to the chosen dimensionality reduction technique (PCA vs. t-SNE), suggesting that the defense requires precise hyperparameter tuning to remain reliable across different layers.

To address these challenges, our next steps involve fully integrating the DLG and AC pipelines into the **Flower framework** to simulate real-time "on-the-fly" detection during federated aggregation. We also plan to evaluate the defense's robustness against **Differential Privacy (DP)** and gradient compression, as adding noise to gradients may inadvertently mask the backdoor triggers that AC relies on for clustering. Finally, scaling the experiments to more complex architectures like ResNet will help determine if the observed activation patterns hold true in high-dimensional feature spaces typical of production-level federated learning systems.

9. Individual Contribution Summary

- Sukirat Singh Dhillon → Federated Learning environment using Flower
- Simon Kraft → Implement first experiment: backdoor attack → model training → DLG reconstruction → AC evaluation.
- Nazanin Parvizi → DLG reconstruction → backdoor attack → model training → AC evaluation.